

Software Performance Engineering

Recitation 2.2

Sophia Sun
Tuesday, October 28, 2025



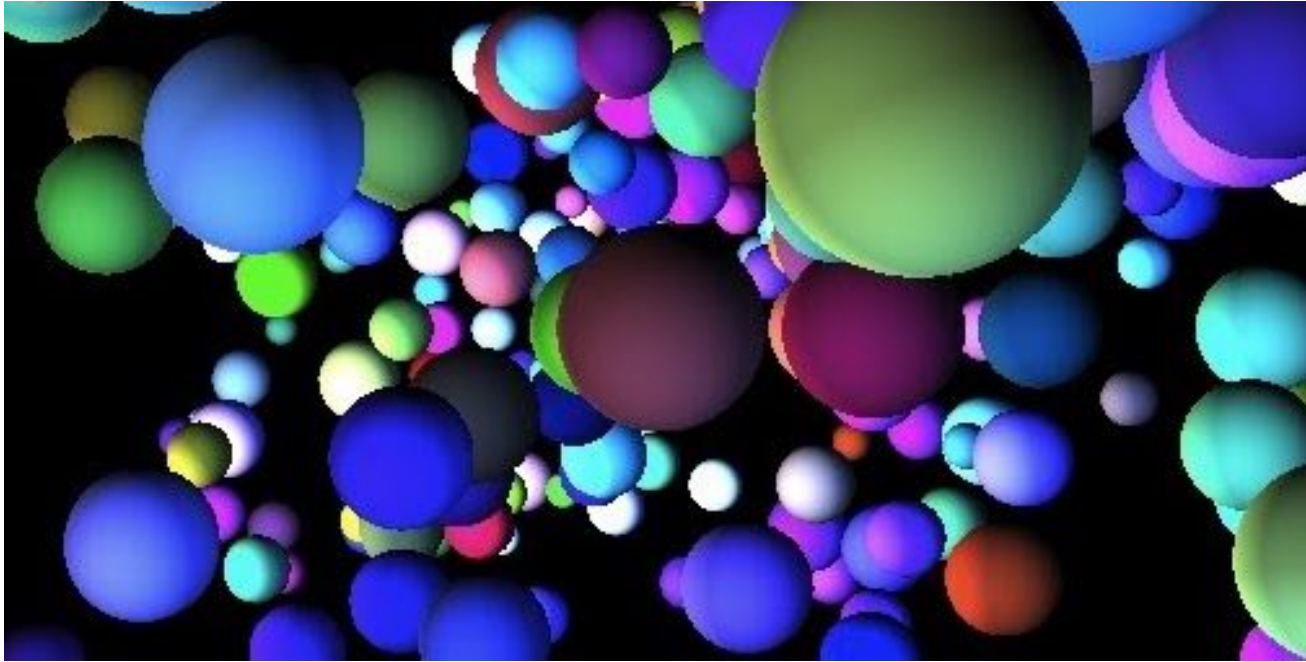
Some due dates

- Team contract of project 2 is due **tonight**.
- Remember to include your teammate in the submission.
-
- Homework 5 will release this Thursday, Oct 30.
- Project 3A and 3B will release this week too.
 - If you're an honors student, you've signed up for this.
 - If you're not an honors student, but want to get some extra credit to make up for your total grade, let me know.
 - The due date of project 3 is December 8 (Monday of final week).

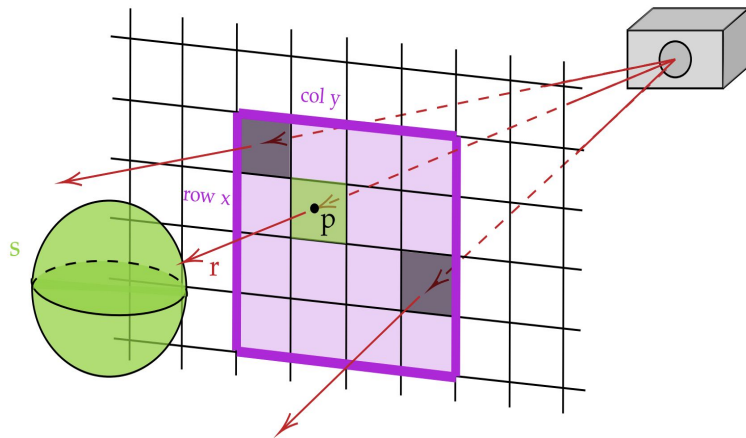
CONTINUE ON PROJECT 2



Project 2: N body simulation

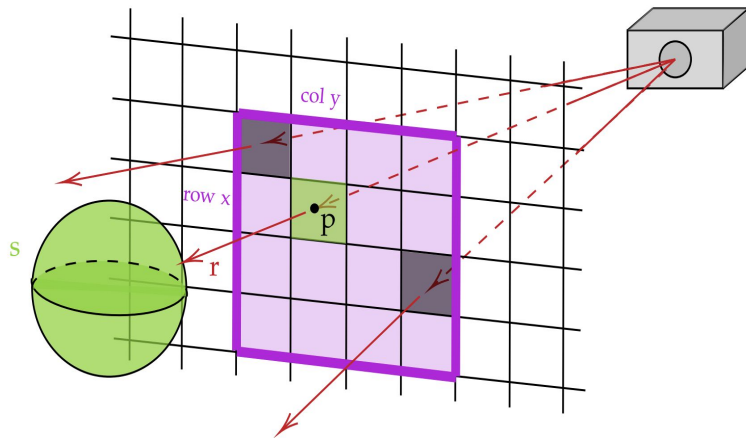


How do we get the pixel value at I and J?



- Given an image pixel location (I, J)
- Find the point in the 3D plane that corresponds to this pixel location (calculate ray from the eye)
- Determine if the ray hits a sphere
 - Complicated math

A different way?

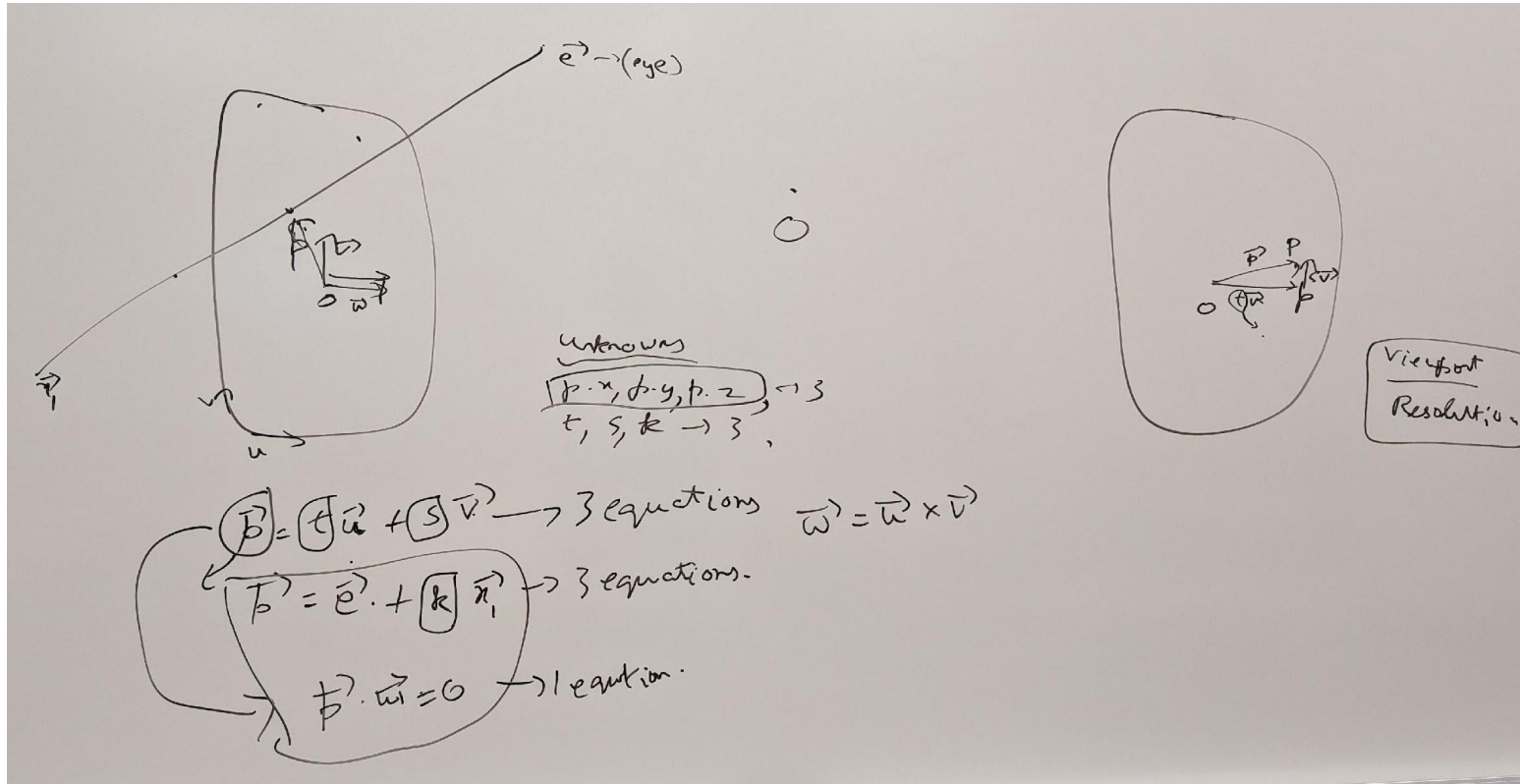


- Given a sphere location, get 8 points making a bounding cube
 - Ex: Given r and (x, y, z) some corners are $(x+r, y+r, z+r)$, $(x-r, y-r, z-r)$, $(x+r, y-r, z-r)$
- Project those points onto the plane
- For the bounding box of the cube in the image, check if the points intersect the sphere
 - If so, color the pixel

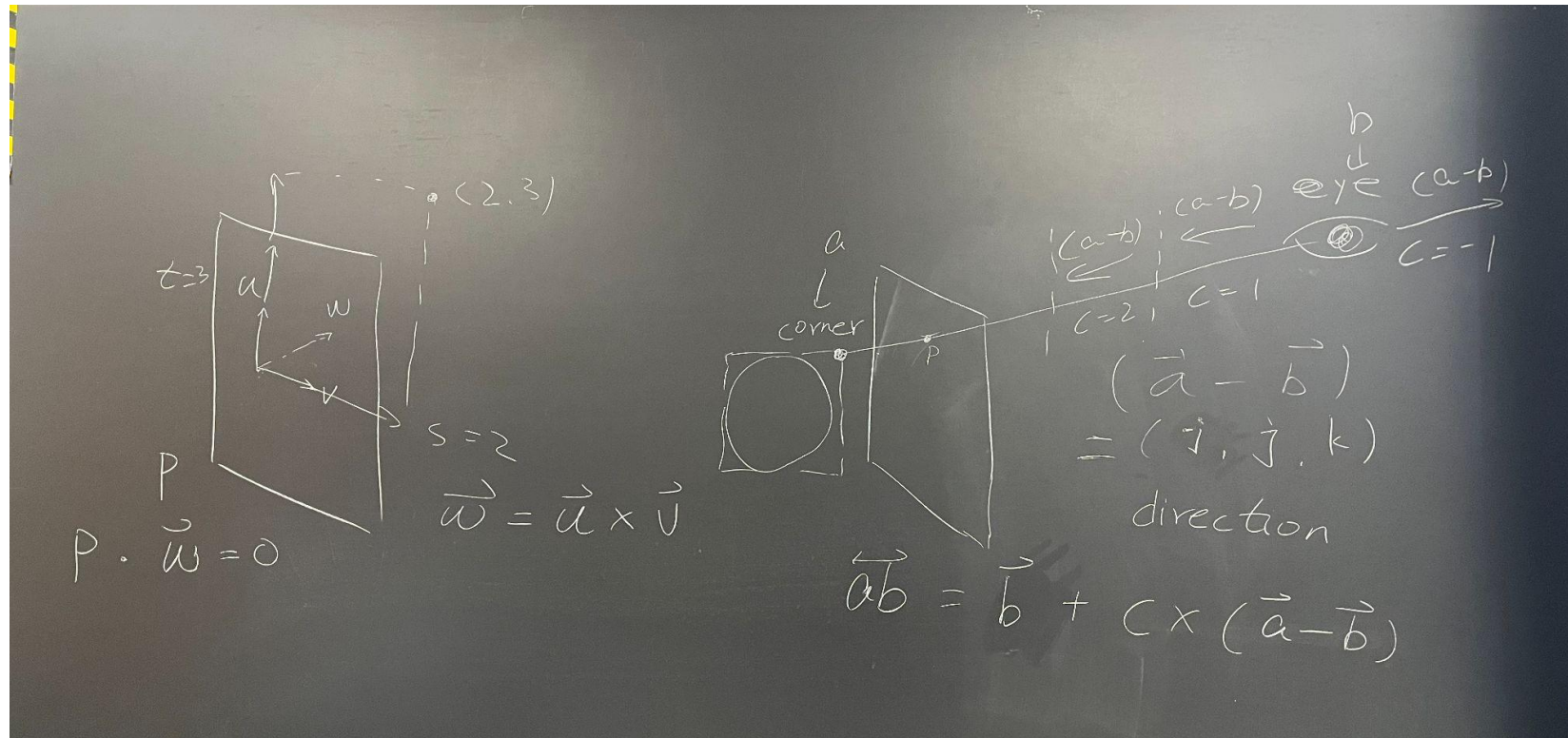
Project the cube onto the plane

- Given points a, b find where they intersect a plane through the origin with orthonormal vectors u, v
- $\overrightarrow{ab} = \vec{b} + c * (\vec{a} - \vec{b})$ for any number c
- Plane = $t\vec{u} + s\vec{v}$ for any numbers t, s
 - Also given by $p \cdot \vec{w} = 0$ where $\vec{w} = \vec{u} \times \vec{v}$
- 3 unknowns, 3 equations (one each for x, y, z components)
- Solve to get the 8 points and get a bounding box of those 8 points
- You can then have a bigger bounding box with just 4 points

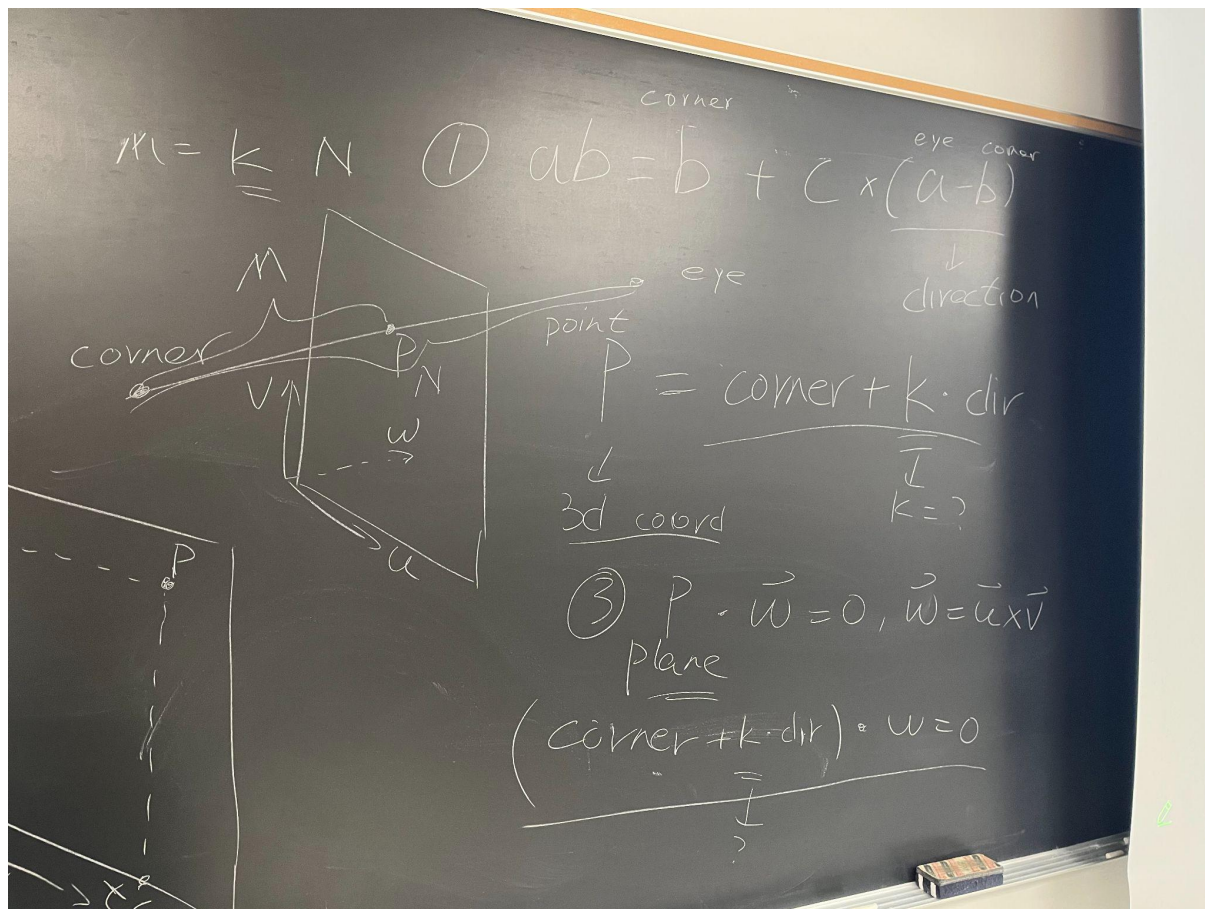
Vector Algebra for Rendering



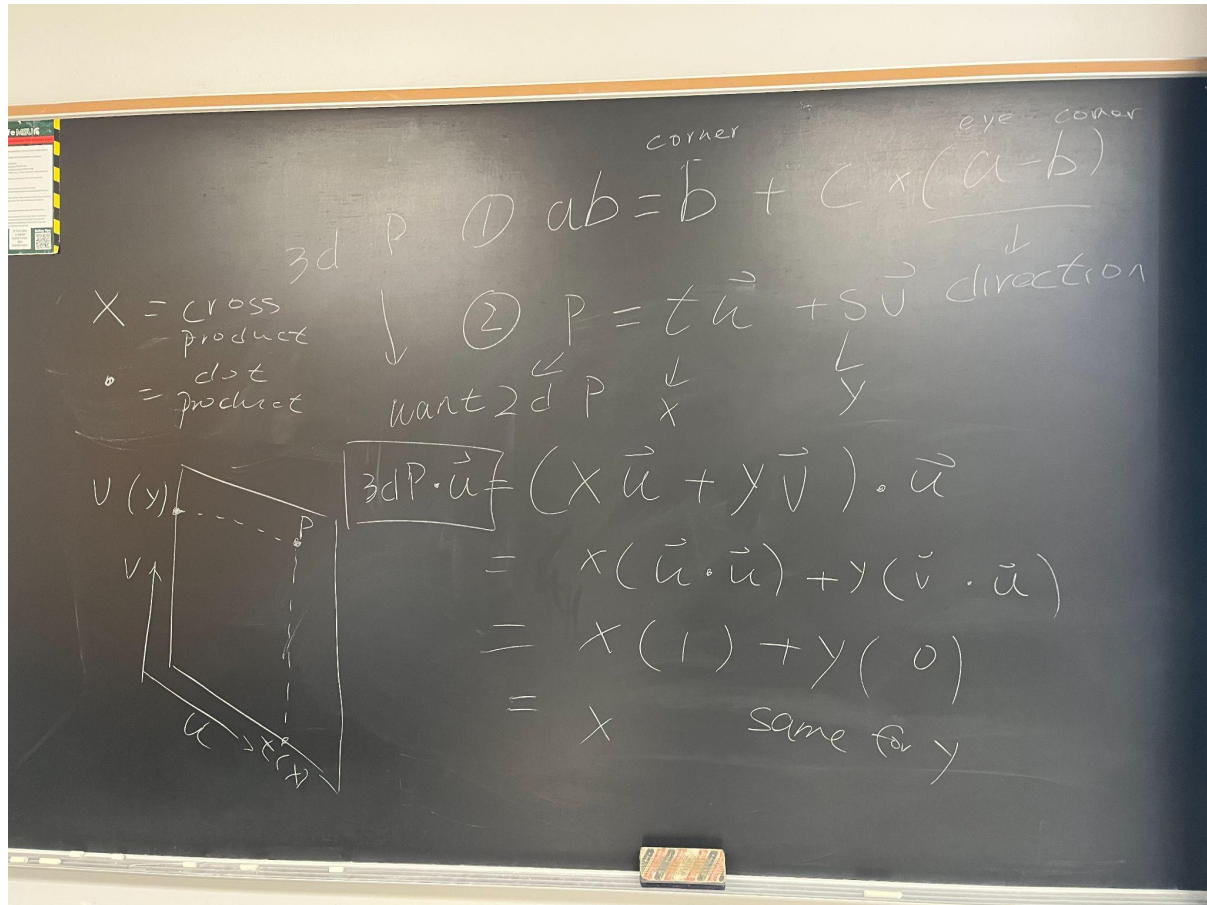
Vector Algebra for Rendering



Vector Algebra for Rendering



Vector Algebra for Rendering



Other Possible Render Optimizations

- Avoid redundancy in light diffusion on material calculations (some part is fixed for each sphere)
- Use quicksort or other efficient sorting algorithms to sort spheres
- Pre-compute origin_to_pixel values. (Identifying large amount of work happening on adjacent values can then be vectorized)

Some Possible Simulate Optimizations

- Redundant collision checks and force of gravity computations
- Try having a bounding box for collision of a sphere over an interval – If nothing intersects this box, the sphere does not collide with anyother sphere for the current iteration of simulation

Parallelization

- Render – Easy to parallelize pixel wise computation (the majority of work)
- Simulate – Can have a race condition!

Force calculation with redundancy

```
for (int i = 0; i < N; i++) {  
    for (int j = 0; j < N; j++) {  
        if (i != j) force[i] += f(i, j);  
    }  
}
```

Force calculation without redundancy

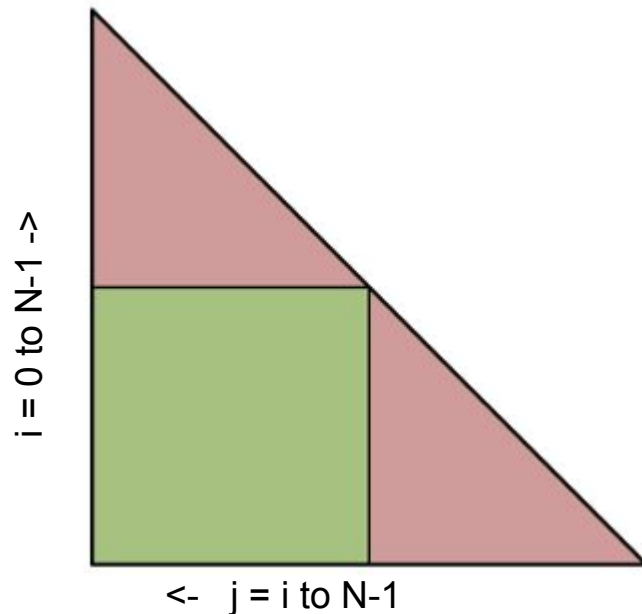
```
for (int i = 0; i < N; i++) {  
    for (int j = i+1; j < N; j++) {  
        force = f(i, j);  
        force[i] += force;  
        force[j] -= force;  
    }  
}
```

Parallelization of Simulate

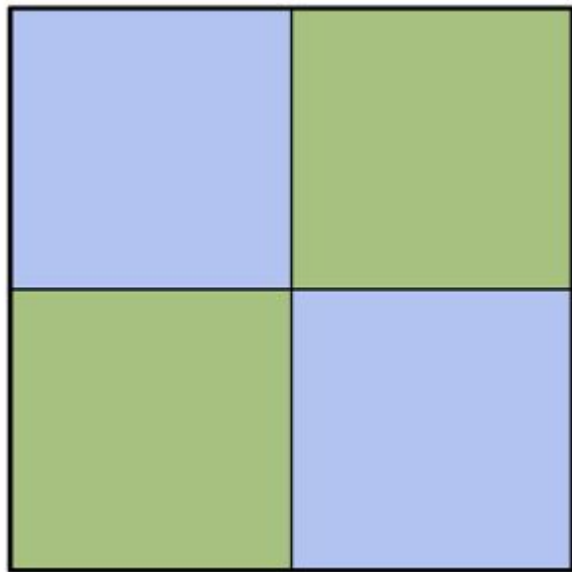
- Iteration space of :

```
for (int i = 0; i < N; i++) {  
    for (int j = i+1; j < N; j++) {  
        // ...  
    }  
}
```

Triangles can be run in parallel.
Rectangle should be run
before/after the triangles.



Parallelization of Simulate - 2



When executing for the rectangle, it can further be parallelized (diagonally opposite ones execute together)

Project 2 Guidelines

- **Tier for Guaranteed B in Beta : 47**
- Your code must match the behavior of the original code exactly!
 - Beware of the lack of floating point associativity!
 - No -ffast-math
- No determinacy races (including benign races)
 - Cilksan should report no races
- Intrinsic for atomic operations and locks are not allowed
 - Ex. the compare-and-swap intrinsic

CODING TIME!

